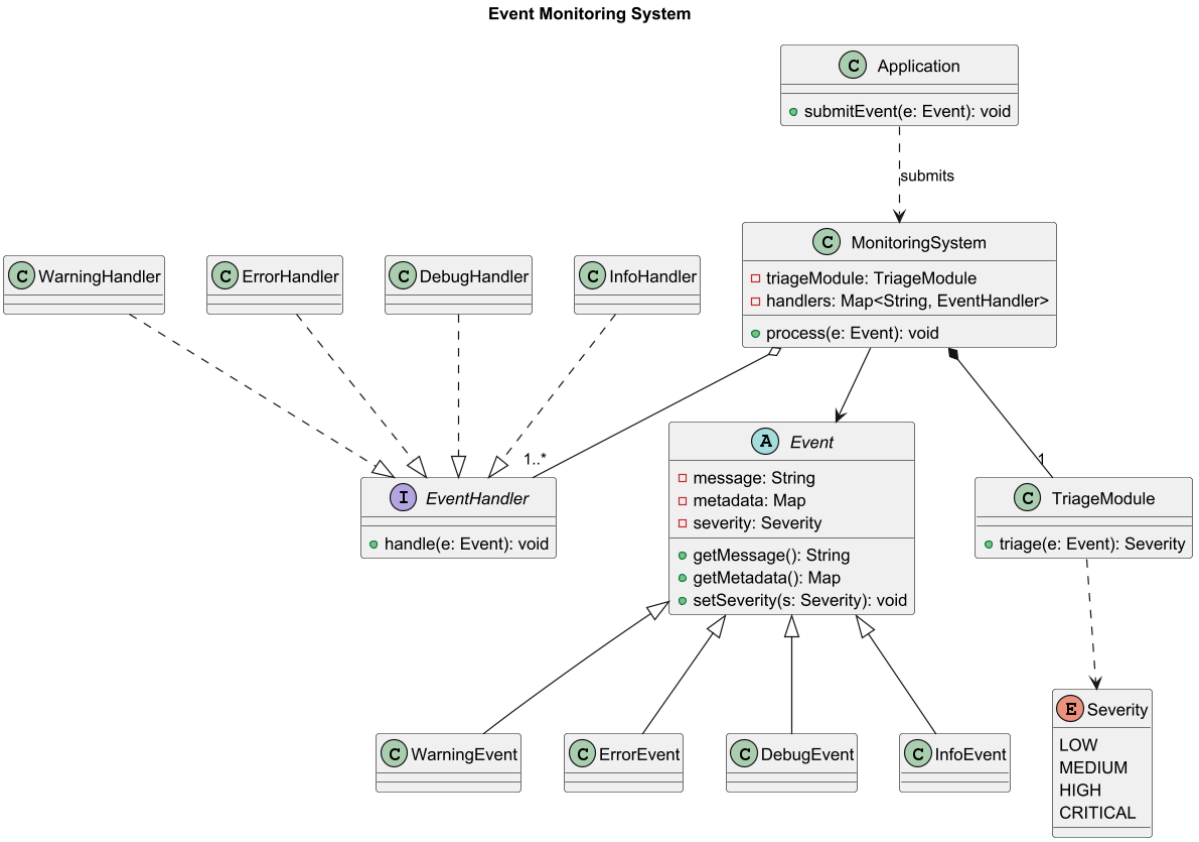
Exercise 1

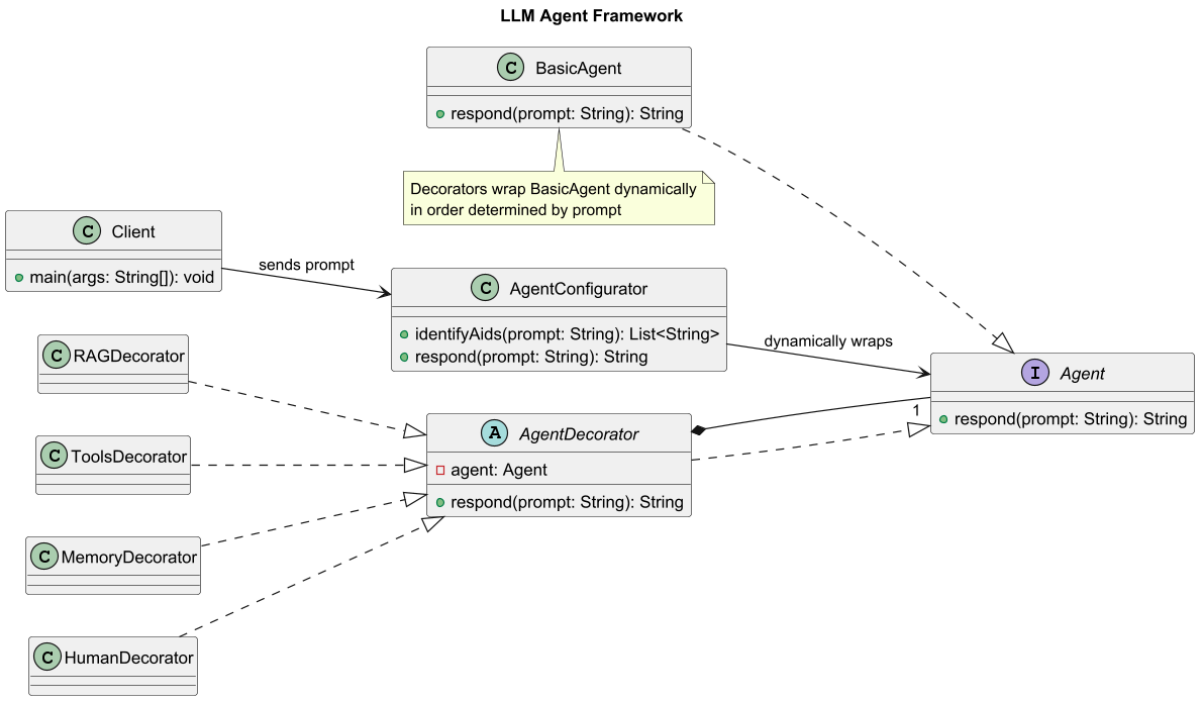


1.
2. The one SOLID principle that must be guaranteed in the design is the Open/Closed Principle, which states that software entities should be open for extension but closed for modification. The problem description explicitly states that as the cloud infrastructure grows, new types of events may arise, meaning the system must be able to support additional event types without requiring changes to existing code. This design guarantees this principle through abstraction and polymorphism. The design has an abstract event class that allows new event types to be introduced through inheritance without modifying existing classes. Similarly, the EventHandler interface allows new handlers to be added by implementing the interface rather than changing the MonitoringSystem. The MonitoringSystem interacts with handlers through the abstraction (EventHandler) rather than concrete classes.

Exercise 2

1. The most suitable design pattern for this system is the Decorator Pattern, with Agent defined as an interface to provide a common contract (respond(prompt)) for all agents. The BasicAgent implements Agent and provides the core LLM response. The AgentDecorator abstract class also implements Agent and wraps another agent, allowing concrete decorators RAGDecorator, ToolsDecorator, MemoryDecorator, and HumanDecorator to extend behavior dynamically without modifying the original class.

Also, the AgentConfigurator handles prompt-specific logic, dynamically wrapping the BasicAgent with the required decorators for each request, such as combining RAG and tools or tools and memory. Finally, the Client sends prompts to the configurator, which ensures that the agent is configured with all necessary capabilities before generating a response.

2. 

3. Zip folder attached